

Comprehensive Technical Documentation: VisionCheckAI Codebase Implementation

1. Introduction and Project Objective

The fundamental objective of the `VisionCheckAI.Server` codebase is to provide a robust API backend for a quality control system. In product-based companies, integrating AI into existing software stacks is traditionally a complex, labor-intensive process. This project seeks to entirely eliminate friction by providing a C# ASP.NET Core API that orchestrates database transactions, user authentication, and Python-based machine learning inference automatically. The code used for this project is localized within the `backend/VisionCheckAI.Server` directory on the GitHub repository.

2. Program Architecture and Configuration Rationale

2.1 Why `Program.cs` structure?

When setting up a modern .NET 8/7 application, there is an inherent trade-off between boilerplate code and explicit configuration. The codebase utilizes the Minimal Hosting Model, which requires minimal computational resources and file overhead to bootstrap the application.

`Program.cs` acts as the highly advanced, state-of-the-art entry point designed primarily for dependency injection and request pipelining. It registers controllers and configures JSON serialization to be case-insensitive. While immensely powerful, configuring CORS and JWT authentication requires precise parameter tuning. The application registers `IAuthService` and `IIInferenceService` via Dependency Injection using `AddScoped`, ensuring that memory is managed efficiently per HTTP request. Furthermore, the `VisionCheckDbContext` is initialized with SQLite, utilizing a physical file path (`visioncheck.db`) generated dynamically from the `ContentRootPath`.

2.2 Architectural Parameters (Database Initialization)

We employed Entity Framework Core with initial seed data. This pre-trained data is crucial here, as the foundational tables already possess the ability to recognize rudimentary users and products without manual database population.

The database initialization is executed inside a service scope: *

`db.Database.EnsureCreated()` guarantees the SQLite file and schema exist. * Inside

`VisionCheckDbContext.cs`, the `OnModelCreating` method is customized. It seeds 3 default users (Admin, Supervisor, Inspector) and 3 active products, mapping to precisely defined entity classes (e.g., M8 Hex Steel Nut).

3. Codebase Division and Data Flow

The codebase is structured into distinct folders representing the fine-grained classes of the application architecture. To evaluate the request flow effectively, the data is split into multiple logical layers.

Folder	File Count	Primary Responsibility	Broad Category
Controllers/	4	Handles HTTP routing, input validation, and HTTP responses.	Routing
Data/	2	Contains <code>VisionCheckDbContext</code> and the <code>Entities/</code> subfolder.	Persistence
Models/	1	Contains <code>DtoModels.cs</code> for data transfer objects (DTOs).	Data Transfer
Services/	2	Contains <code>AuthService.cs</code> and <code>InferenceService.cs</code> .	Business Logic

4. Comprehensive Controller Pipeline

The API strategy was separated into specific controller classes. This phased approach is a best practice in RESTful API design.

4.1 Stage 1: AuthController and Security (Login Training)

In the first stage, the parameters of the user's login request are processed by `AuthController`. * **Auto-Provisioning**: If a user is not found in the database, the controller creates a new `UserEntity`. It applies a regularization technique by checking substring matches (e.g., "admin" in username assigns the "Administrator" role, preventing

unclassified users). * **JWT Generation**: It calls `IAuthService.GenerateJwtToken`, which signs a token using a `SymmetricSecurityKey` and `HmacSha256`, appending claims like `ClaimTypes.Role` for subsequent endpoint authorization.

4.2 Stage 2: InspectionsController and File Fine-Tuning

After the authentication is validated, the `InspectionsController` handles the heavy lifting of image ingestion. * **File Handling**: Accepts `[FromBody] IFormFile file`, generates a GUID-based filename, and writes it asynchronously to the local `wwwroot/uploads` directory. * **Database Graph**: Creates an `InspectionEntity` and, if a defect is found, appends `DefectEntity` records with initialized bounding box coordinates (`BboxX`, `BboxY`, etc.) to the `entity.Defects` collection before calling `SaveChangesAsync()`. * **Human Review**: The `ReviewInspection` endpoint allows supervisors to modify `ReviewStatus`, `ReviewNotes`, and `CorrectedResult`, essentially providing a manual override mechanism to correct misclassifications from the AI.

5. Detailed Inference Service Implementation

Evaluating an industrial classification model from a backend requires more than just an HTTP call; it requires robust inter-process communication.

5.1 PyTorchInferenceService Explained

- **ProcessStartInfo**: The `PyTorchInferenceService` uses `System.Diagnostics.Process` to execute the external Python script. It points to `predict.py` in the parent directory.
- **Asynchronous I/O**: It utilizes `ReadToEndAsync()` on both standard output and standard error, preventing the .NET thread pool from becoming blocked while the Python ML model processes the image in the background.
- **JSON Deserialization**: The raw string output from the Python model is parsed into an `InferenceResult` object using `System.Text.Json.JsonSerializer`.

5.2 The Fallback Function (Deep Dive into the Confusion Matrix of Errors)

To operationalize this service on a factory floor when Python is missing, a dedicated `FallbackInference` function is established. * **Purpose**: This function abstracts away all the complexity if the Python script fails. It accepts a single raw image file path,

automatically applies string matching on the filename (checking for "rust", "scratch", "deform", "fracture"), and outputs simulated confidence percentages (e.g., 0.96 for Rusting). This single-line fallback is the literal bridge between theoretical ML environments and practical, uninterrupted API availability.

6. Real-World Dashboard Aggregation

6.1 Analyzing LINQ Queries

By visually inspecting `DashboardController.cs`, we see the backend divides the database results into actionable insights to be tracked:

- * **Group By Operations:** The controller accurately assigns defect counts by utilizing `GroupBy(d => d.Category)` and `GroupBy(d => d.Severity)`, projecting them into `CategoryCountDto` objects.
- * **Time-Series Analysis:** A rolling 7-day query is constructed by taking `DateTime.UtcNow.Date.AddDays(-6)` and iterating `Enumerable.Range(0, 7)` to calculate daily totals, passes, and defects, allowing the client frontend to confidently chart the manufacturing pipeline's health.

7. References

1. System.Diagnostics.Process Class Documentation:
<https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.process>
2. ASP.NET Core Authentication and Authorization:
<https://learn.microsoft.com/en-us/aspnet/core/security/>